TITAN: A Hardware-Accelerated NTT Engine for Post-Quantum Cryptography on ARM64 NEON and AVX2

Andrea Catino
Independent Researcher, Italy
https://github.com/catinoandrea-lgtm/TITAN-PQC

April 2026

Abstract

We present **TITAN**, an optimized implementation of the ML-KEM-512 Key Encapsulation Mechanism (FIPS 203) featuring a high-performance Number Theoretic Transform (NTT) engine targeting ARM64 NEON and x86_64 AVX2 architectures. The core contribution is a hybrid NEON/scalar NTT with 16-bit Montgomery multiplication and Barrett reduction at every butterfly level. We evaluate TITAN on five hardware platforms and compare against the *pqcrystals/kyber* reference implementation. On identical hardware (GitHub Actions x86_64), TITAN's scalar path is **1.11× faster** than the reference C implementation (121.64 μs vs. 135.31 μs for a full handshake). With ARM NEON acceleration, TITAN achieves **31.60 μs** on Apple M1 (**4.28**× faster than reference C) and **88.79 μs** on Samsung Galaxy S22 Ultra. A 9-test KAT conformance suite produces a deterministic cross-platform fingerprint, verified identical on x86_64 and ARM64. All source code is available under MIT license.

1 Introduction

The standardization of ML-KEM (FIPS 203) [1] and ML-DSA (FIPS 204) by NIST has created an urgent need for efficient implementations across diverse hardware platforms. While considerable effort has been invested in optimizing these algorithms for server-class x86 processors and dedicated hardware accelerators, the mobile and IoT ecosystem—representing the majority of deployed ARM64 devices—remains underserved.

The Number Theoretic Transform (NTT) is the computational bottleneck of all lattice-based cryptographic schemes. For ML-KEM-512, operating over $R_q = \mathbb{Z}_q[X]/(X^{256}+1)$ with $q = 3329$, the NTT converts polynomial multiplication from $O(n^2)$ to $O(n \log n)$. Optimizing the NTT directly translates to faster key generation, encapsulation, and decapsulation.

We present TITAN, a multi-architecture NTT engine and ML-KEM-512 implementation with the following contributions:

(1) **A hybrid NEON/scalar NTT architecture** that uses 8-wide ARM NEON SIMD for butterfly levels with stride ≥ 8, falling back to scalar with Barrett reduction for the final levels, avoiding register pressure on narrow strides.

(2) **A 16-bit Montgomery multiplication core** using `vmull` + `vmlsl` + `vshrn` (6 NEON instructions per 8 elements), with compile-time constants derived via Hensel lifting.

(3) **Measured speedup over the reference implementation:** 1.11× on scalar C (same hardware), 4.28× with NEON on Apple M1, 1.52× on Samsung S22 Ultra.

(4) A 9-test KAT conformance suite with a deterministic cross-platform fingerprint, verified identical on x86_64 and ARM64.

2 Background

2.1 ML-KEM (FIPS 203)

ML-KEM is a lattice-based KEM standardized as FIPS 203 [1]. ML-KEM-512 uses module rank $k = 2$, modulus $q = 3329$, and secret noise distribution CBD($\eta = 2$). The public key is 800 bytes, secret key 1632 bytes, ciphertext 768 bytes, and shared secret 32 bytes. KeyGen accepts 64 bytes of randomness ($d \| z$) per FIPS 203 §7.1.

2.2 Number Theoretic Transform

The NTT operates modulo $q = 3329$ ($q \equiv 1 \pmod{256}$), enabling a 128-point Cooley–Tukey [4] forward and Gentleman–Sande [5] inverse transform. The primitive 256th root of unity $\zeta = 17 \pmod{3329}$ generates the twiddle factors, pre-stored in Montgomery form.

2.3 Montgomery Multiplication

Montgomery multiplication [3] computes $a \cdot b \cdot R^{-1} \bmod Q$ without division. For $R = 2^{16}$ and $Q = 3329$:

$$Q_{\text{INV}} = -3327 \pmod{2^{16}} \tag{1}$$

$$R^2 \bmod Q = 1353 \tag{2}$$

$$N^{-1} \cdot R \bmod Q = 512 \tag{3}$$

Barrett reduction with $v = 20159 \approx 2^{26}/Q$ bounds coefficients after each butterfly.

3 TITAN Architecture

3.1 Hybrid NEON/Scalar NTT

The forward NTT processes 7 butterfly levels. The first 5 levels (stride 128 down to 8) use ARM NEON 8-wide SIMD: each butterfly loads two `int16x8_t` vectors, computes the twiddle multiplication via `fqmul_neon`, and stores the sum and difference. A Barrett reduction pass is applied after each NEON level. The final 2 levels (stride 4 and 2) use scalar arithmetic with Barrett at every butterfly. This avoids data reorganization overhead at sub-8-element strides.

The inverse NTT mirrors this structure: scalar Gentleman–Sande for stride 2 and 4, NEON for stride 8 through 128, with final normalization by $N^{-1} \cdot R \bmod Q = 512$.

3.2 Montgomery Core (NEON)

The `fqmul_neon` function computes 8 modular multiplications in 6 instructions: `vmull_s16` (2×, low/high halves), `vmovn_s32` + `vmulq_s16` (Montgomery quotient), `vmlsl_s16` (2×, subtraction), `vshrn_n_s32` (2×, final shift). This avoids 32-bit intermediate expansion, keeping all arithmetic within the NEON 128-bit register file.

3.3 Domain Tracking

All polynomial arithmetic operates in Montgomery domain. Conversion gates `poly_tomont` (multiply by R^2) and `poly_frommont` (multiply by 1) are placed at encode/decode boundaries. NTT zetas are pre-stored in Montgomery form, eliminating redundant domain conversions within the NTT/INTT/pointwise chain.

3.4 Streaming Matrix Generation

Matrix **A** is generated via rejection sampling from SHAKE-128. TITAN uses incremental squeeze (168 bytes/block) until 256 valid coefficients are obtained, guaranteeing termination for all inputs. This is more robust than fixed-buffer approaches and more cache-friendly than byte-at-a-time streaming.

3.5 Constant-Time Decapsulation

The IND-CCA2 decapsulation uses Fujisaki–Okamoto implicit rejection. Ciphertext comparison uses bitwise OR accumulation (`diff |= a[i] ^ b[i]`), followed by arithmetic masking to select between the derived key and the rejection seed z. No branches depend on secret data.

4 Implementation

TITAN is implemented in C99 with platform-specific SIMD intrinsics. The ARM NEON KEM (`kem.c` + `ntt32.c` + `keccakf1600.c`) is approximately 900 lines; the scalar fallback (`kem32.c`) replicates the same protocol without SIMD; the AVX2 math engine (`titan_avx2.c`) is 160 lines. Single-file variants are provided for CxxDroid (Android) and a-Shell (iOS).

FIPS 203 conformance: KeyGen accepts 64 bytes ($d\|z$); all symmetric operations use SHA3-256, SHA3-512, SHAKE-128, SHAKE-256 via Keccak-f[1600] [10]; z is stored directly in the secret key for implicit rejection. A universal Makefile auto-detects architecture (ARM64/Apple/x86_64) and selects the appropriate build.

5 Experimental Results

5.1 Platforms

Table 1: Evaluation platforms. All benchmarks compiled with `-O3`.

Platform	CPU	Arch	Freq	Compiler	SIMD
Apple M1 (GitHub CI)	Firestorm	ARM64	3.2 GHz	Clang 15	NEON
Samsung S22 Ultra	Cortex-X2	ARM64	3.0 GHz	Clang 9	NEON
Android mid-range	A55/A78	ARM64	2.0 GHz	Clang 9	NEON
GitHub CI x86	Xeon/EPYC	x86_64	∼2.8 GHz	GCC 13	(scalar)
AMD EPYC 7742	Zen 2	x86_64	2.25 GHz	GCC 11	AVX2

5.2 Baseline Comparison (Scalar)

Table 2 compares TITAN's scalar C99 implementation against the *pqcrystals/kyber* reference C implementation on identical hardware (GitHub Actions Ubuntu x86_64). Both compiled with `gcc -O3` without SIMD. $N = 10{,}000$ iterations with 100-iteration warmup.

TITAN's scalar implementation is 11% faster overall, with the largest gains in decapsulation (22%) and key generation (18%). The encapsulation is marginally slower (0.95×) due to streaming SHAKE-128 matrix generation overhead compared to the reference's bulk approach. The net advantage demonstrates that TITAN's optimizations extend beyond SIMD—the streaming architecture, Barrett placement, and Montgomery domain tracking contribute measurable improvements even in pure scalar code.

Table 2: Baseline comparison on GitHub Actions x86_64. Ratio > 1.0 = TITAN faster.

Operation	**TITAN scalar (μs)**	**pqcrystals ref (μs)**	**Ratio**
Keygen	30.23	35.76	**1.18**×
Encaps	44.78	42.49	0.95×
Decaps	46.62	57.07	**1.22**×
Total	**121.64**	**135.31**	**1.11**×

5.3 NEON-Accelerated KEM Performance

Table 3 presents the full ML-KEM-512 protocol benchmarks with ARM NEON acceleration. The rightmost column shows speedup relative to the *pqcrystals* reference C (135.31 μs on the same CI hardware).

Table 3: NEON-accelerated ML-KEM-512 ($N = 1000$). Speedup vs. pqcrystals ref C scalar.

Platform	**Keygen**	**Encaps**	**Decaps**	**Total (μs)**	**vs. Ref**
Apple M1	8.13	11.55	11.92	**31.60**	**4.28**×
Samsung S22 Ultra	26.16	34.04	28.60	**88.79**	**1.52**×
Android mid-range	63.87	84.88	94.68	243.43	0.56×

Apple M1 achieves a 4.28× speedup over the reference, completing a full handshake in 31.60 μs. The Samsung S22 Ultra (Cortex-X2) achieves 1.52× at 88.79 μs—a sub-100 μs handshake on a mobile phone. The mid-range device is slower than the reference scalar due to its lower clock speed (2.0 GHz) and narrower microarchitecture (Cortex-A55 efficiency cores), but still completes the handshake in under 250 μs.

5.4 NTT Engine Performance

Table 4: Isolated NTT engine benchmarks (μs).

Platform	**NTT Fwd**	**NTT Inv**	**Pointwise**	**Keccakx4**
Apple M1 (NEON)	**0.29**	**0.25**	—	—
Samsung S22 Ultra (NEON)	0.87	0.51	—	—
Android mid-range (NEON)	4.15	1.88	—	—
AMD EPYC 7742 (AVX2)	2.40	1.32	0.020	1.36
Google Colab (AVX2)	6.08	1.92	0.016	1.70

A striking result is that the Apple M1 NEON NTT (0.29 μs forward) is 8.3× faster than the EPYC AVX2 NTT (2.40 μs) and 21× faster than Google Colab AVX2. The Samsung S22 Ultra (0.87 μs) also outperforms both x86 platforms. This advantage arises from the M1's wide-issue microarchitecture with 4 NEON execution units, low-latency L1 cache, and the compact 16-bit data format that maximizes elements per vector register.

5.5 KAT Conformance

The TITAN KAT suite comprises 9 tests: SHA3-256, SHA3-512, and SHAKE-128 against NIST known answer vectors; NTT forward-inverse roundtrip; FIPS 203 key size verification; KEM determinism and correctness; implicit rejection (modified ciphertext); and an accumulated test

of 100 full KEM cycles with deterministic seeds. The accumulated test produces a SHA3-256 fingerprint of all XOR-combined shared secrets:

```
807c873a3ad4a624b16bfcd44018d8cf0949da547561fe99daf5039cdf1342bd
```

This fingerprint has been verified identical on x86_64 Ubuntu (GCC 11), GitHub Actions Ubuntu (GCC 13), Android ARM64 Samsung S22 Ultra (Clang 9), and Android ARM64 mid-range (Clang 9), confirming cross-platform determinism and protocol correctness.

6 Related Work

The *pqcrystals/kyber* reference [2] provides portable C and optimized AVX2 implementations. Our baseline comparison (Table 2) shows TITAN's scalar path is 1.11× faster than the reference C on the same hardware, establishing that the performance gains are not solely attributable to SIMD but also to architectural choices in the code structure.

The *mlkem-native* project [6] provides speed-optimized implementations targeting multiple platforms. Becker et al. [8] demonstrated vectorized NTT for ARM Cortex-A, achieving competitive AVX2 performance. Stelzer et al. [7] investigated ISA extensions for OpenTitan's OTBN accelerator. TITAN differentiates itself through its focus on practical mobile deployment (tested on actual Android devices via CxxDroid), its hybrid NEON/scalar approach, its single-file compilation model, and its comprehensive KAT suite with a cross-platform deterministic fingerprint.

7 Future Work

Several directions are planned. First, validation against NIST ACVP test vectors will provide formal FIPS 203 conformance certification. Second, the NTT engine will be extended to ML-KEM-768 and ML-KEM-1024 by adjusting the module rank while reusing the polynomial arithmetic core. Third, SVE2 (Scalable Vector Extension 2), available on recent ARM cores (Cortex-X3+), offers wider vectors with predication and could further accelerate the NTT. Additionally, we are exploring alternative polynomial compression techniques based on telescoping series representations, and the reusability of the NTT engine for ML-DSA (FIPS 204).

8 Conclusion

We presented TITAN, a hardware-accelerated NTT engine and ML-KEM-512 implementation that achieves **4.28× speedup** over the *pqcrystals* reference on Apple M1 and **sub-100 μs handshakes** on flagship mobile devices. Even TITAN's scalar path outperforms the reference by 11%, demonstrating that the performance gains derive from architectural choices beyond SIMD: streaming SHAKE-128 matrix generation, Barrett reduction at every butterfly, and explicit Montgomery domain tracking. A 9-test KAT suite with a deterministic cross-platform fingerprint provides reproducible correctness verification. These results demonstrate that post-quantum key exchange is practical for real-time mobile applications today, without dedicated hardware accelerators.

References

[1] NIST, "Module-Lattice-Based Key-Encapsulation Mechanism Standard," FIPS 203, August 2024.

[2] R. Avanzi et al., "CRYSTALS-Kyber: Algorithm Specifications and Supporting Documentation," NIST PQC Round 3 Submission, 2021.

[3] P. L. Montgomery, "Modular Multiplication Without Trial Division," *Mathematics of Computation*, vol. 44, no. 170, pp. 519–521, 1985.

[4] J. W. Cooley and J. W. Tukey, "An Algorithm for the Machine Calculation of Complex Fourier Series," *Mathematics of Computation*, vol. 19, no. 90, pp. 297–301, 1965.

[5] W. M. Gentleman and G. Sande, "Fast Fourier Transforms: For Fun and Profit," *Fall Joint Computer Conference*, pp. 563–578, 1966.

[6] pq-code-package, "mlkem-native: Speed-optimized ML-KEM implementation," GitHub, 2024.

[7] F. Stelzer et al., "Enabling Lattice-Based Post-Quantum Cryptography on the OpenTitan Platform," in *ASHES Workshop*, ACM CCS, 2023.

[8] H. Becker et al., "Neon NTT: Faster Dilithium, Kyber, and Saber on Cortex-A72 and Apple M1," *IACR TCHES*, 2022.

[9] ARM Ltd., "ARM Architecture Reference Manual: ARMv8-A," 2024.

[10] NIST, "SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions," FIPS 202, August 2015.

A Availability and Reproducibility

Source code, benchmarks, KAT suite, and baseline comparison script are available at https://github.com/catinoandrea-lgtm/TITAN-PQC under the MIT License. GitHub Actions CI runs all tests automatically on every push. A Google Colab notebook provides instant AVX2 testing without local setup.